

CryptWill — Complete Project Documentation

For AI Coding Agent (Cursor / Claude Code / Windsurf)

Team: Semicolon Survivors | Confluence 2.0

This file is the single source of truth for building CryptWill. Read every section before writing any code.

SECTION 1 — PRD (Product Requirements Document)

App Identity

- Name: CryptWill
- Tagline: Your crypto shouldn't die with you.
- Category: Web3 / LegalTech / FinTech
- Blockchain: Stellar (Testnet → Mainnet)
- Theme: Dark, premium, trustworthy. Think Linear meets Coinbase.

The Problem

\$140 billion in cryptocurrency is permanently lost because owners died and nobody had the private key. No court can recover it. No lawyer can enforce it. 106 million Indians own digital assets right now with zero inheritance plan. Every existing solution fails: paper wills expose your key, centralized services require trusting a company, telling family means they access funds before death. The gap is real, massive, and unsolved.

Target Users

Owner (Primary)

Crypto holders aged 25-55 who own meaningful digital assets and believe in self-custody. They want peace of mind that their family won't lose everything when they die.

Beneficiary (Secondary)

Family members named as recipients. They need a simple read-only portal to see what they're set to receive, and automatic notification + transfer when inheritance executes.

Guardian (Secondary)

Trusted people (spouse, sibling, lawyer) who act as human verification before inheritance triggers. Simple vote interface — approve or deny.

Enterprise / Legal (Tertiary)

Law firms and judiciary teams managing client digital estates. Full dashboard, compliance exports, multi-client management.

Core Features

1. Registration & Identity

- Email + password signup
- 6-digit OTP email verification on signup
- Freighter wallet connection (Stellar browser wallet)
- KYC document upload: Aadhaar / PAN / Passport (encrypted before storage)
- Bank account declaration (estate record only — not for payment processing)
- Terms & Conditions: mandatory acceptance with timestamp + IP logged
- Second T&C acceptance required before smart contract deployment
- Profile management (name, phone, country, DOB)

2. Dead Man's Switch — Core Engine

- Owner sets check-in interval: 14 / 30 / 60 / 90 days (default 30)
- One-click check-in via email link (no login needed — JWT signed)
- Check-in signs a micro Stellar transaction (0.0001 XLM, takes 5 seconds)
- Mints a Proof of Life token on Stellar (timestamped, public, permanent)
- Missed check-in cascade:
 - 5 days before deadline → email reminder
 - 2 days before → email + SMS reminder
 - 12 hours before → final email + SMS (URGENT)
 - Deadline passed → owner alert email + SMS + missed_checkin_count++
 - 3 consecutive misses → guardian notification triggered immediately
- Owner can cancel entire inheritance process with one wallet signature at any time while alive

3. Guardian Consensus System

- Owner nominates 3 to 7 guardians
- Configurable quorum: how many must approve (default 3 of 5)
- Guardian invite flow: email sent → guardian clicks link → creates guardian account → status: Active
- When inheritance triggers (3 missed check-ins):
 - All active guardians get email + SMS immediately
 - Guardian portal shows vote prompt: Approve / Deny with optional note
 - Real-time vote tally visible to all guardians
 - Quorum reached → smart contract executes
 - Single deny + strong reason → 48-hour review hold
 - Vote window: 14 days from notification
- If owner returns alive → one wallet signature cancels everything, all guardians notified

4. Digital Asset Registry

- Register Stellar wallet addresses and XLM holdings
- Register custom Stellar tokens and NFTs
- Register encrypted document references (IPFS-stored vault files)
- Per asset: name, type, estimated value, assigned beneficiary, release day, special instructions

- Multiple beneficiaries per asset with percentage splits
- Assets shown on dashboard with status (Active / Transferred / Pending)

5. Will Builder

- Per-asset inheritance policies
- Staged release timeline:
 - Day 0: beneficiaries notified of inheritance starting
 - Day 30: first asset tranche transfers
 - Day 60: second tranche
 - Day 90: vault file keys released + final assets
- Visual timeline preview before deployment
- Pre-deployment checklist (assets added? beneficiaries verified? guardians active?)
- Deploy to Stellar: Freightier signs → Soroban contract created → address shown
- Download PDF will summary (Pro tier only)

6. Secure Digital Vault

- Upload: PDF, JPG, PNG, DOCX, TXT (max 50MB per file)
- Encryption: AES-256 client-side BEFORE upload (server never sees plaintext)
- Storage: encrypted files on IPFS via Pinata
- Key management: encryption key stored in Soroban smart contract (not in DB)
- Key released to beneficiary ONLY after contract execution
- Assign each file to a specific beneficiary
- Storage limits: 500MB free / 5GB Pro
- File manager: upload, view info, delete, reassign beneficiary

7. Proof of Life NFT Chain

- Every check-in: mint lightweight Stellar asset (custom token)
- Token contains: timestamp, owner wallet, check-in sequence number
- Fully public on Stellar Explorer — anyone can verify
- Dashboard displays last 6 check-ins as visual timeline
- Guardians and beneficiaries can view the chain

8. Shamir's Secret Sharing

- Private key split into 3 shards — 100% client-side in browser
- Shard 1: downloaded by owner (never sent to server)
- Shard 2: emailed to a guardian of owner's choice
- Shard 3: encrypted and stored in Soroban contract
- All 3 shards required to reconstruct — no single party can steal or access early
- Reconstruction happens client-side only

9. Beneficiary Preview Mode

- Beneficiaries create account via email invite link
- They see: asset names, estimated values, their assigned percentage, release timeline
- They see: vault file names (NOT contents)

- They CANNOT access any asset or open any file before execution
- Automatic notification when inheritance process starts
- Assets auto-received when smart contract releases each tranche

10. Smart Contract Executor

- Each user gets their own Soroban contract deployed on Stellar
- Contract stores: check-in interval, guardian addresses, quorum, asset policies, staged timeline
- Triggers only after: 3 missed check-ins confirmed + guardian quorum vote
- Executes staged release automatically on defined schedule
- Every step permanently recorded on Stellar blockchain
- Cannot be overridden by CryptWill team after deployment
- Direct Stellar Explorer link shown on dashboard

11. Subscription Plans

FREE — ₹0/month

- 1 Stellar wallet registered
- Up to 2 beneficiaries
- Up to 3 guardians
- 30-day check-in interval only
- 500MB vault storage
- Email notifications only
- Email reminders 5 days before deadline

PRO — ₹299/month or ₹2,499/year

- Unlimited assets
- Up to 10 beneficiaries
- Up to 7 guardians
- Custom intervals: 14 / 30 / 60 / 90 days
- 5GB vault storage
- Email + SMS notifications
- Reminders 7 days before deadline
- Custom staged release timeline
- PDF will generation
- Priority support

ENTERPRISE — Custom pricing

- Unlimited everything
- Legal team dashboard
- Multi-client management
- Compliance export (PDF + CSV)
- Judiciary access panel
- Full API access
- Dedicated support

12. Terms & Conditions

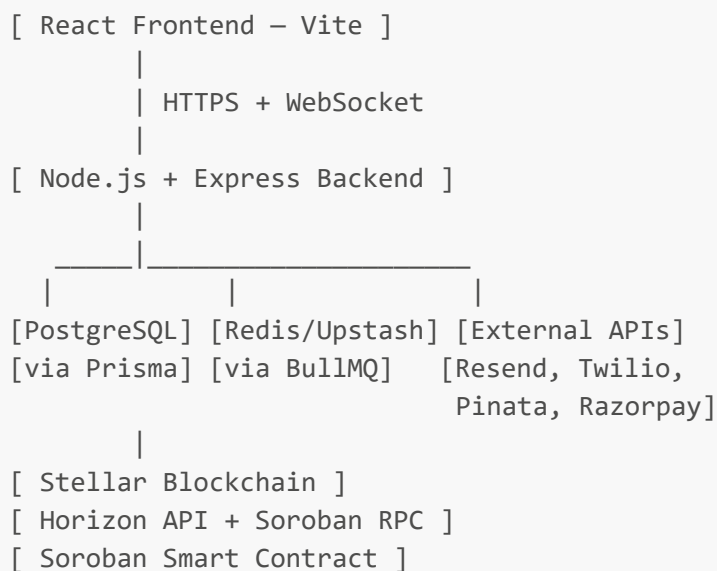
- Full T&C page accessible before signup
- Checkbox on signup (timestamp + IP stored in DB)
- Second mandatory acceptance before smart contract deployment
- Plain-language summary shown above legal text
- Versioned: if T&C updated, existing users must re-accept before next action

Features to Skip for Hackathon MVP

- AI will assistant
- Cross-chain support (Ethereum, Solana)
- Native mobile app
- Death certificate oracle integration
- DAO dispute resolution
- Actual KYC verification service (upload only, no third-party check)
- Hindi / regional language UI
- Family vault (shared planning)

SECTION 2 — TRD (Technical Requirements Document)

Architecture



Frontend Packages

```

react@18
vite@5
tailwindcss@3
react-router-dom@6
  
```

zustand@4	(state management)
react-hook-form@7	(forms)
zod@3	(validation)
stellar-sdk@12	(Stellar blockchain)
@stellar/freighter-api@2	(wallet connection)
socket.io-client@4	(real-time updates)
react-dropzone@14	(file uploads)
crypto-js@4	(AES-256 encryption client-side)
shamir-secret-sharing@1	(key splitting client-side)
axios@1	(HTTP requests)
react-hot-toast@2	(toast notifications)
recharts@2	(charts)
date-fns@3	(date formatting)
lucide-react@latest	(icons)

Backend Packages

express@4	
prisma@5 + @prisma/client@5	
jsonwebtoken@9	(JWT auth)
bcryptjs@2	(password hashing)
bullmq@5	(job queues – CRITICAL for notifications)
ioredis@5	(Redis client)
socket.io@4	(real-time)
resend@6	(transactional email)
twilio@5	(SMS)
multer@1	(file upload handling)
@pinata/sdk@2	(IPFS storage)
express-rate-limit@7	(rate limiting)
express-validator@7	(input validation)
helmet@7	(security headers)
cors@2	
dotenv@16	
stellar-sdk@12	(Stellar server-side)
razorpay@2	(payments)
node-cron@3	(scheduled check-in monitor)
date-fns@3	

Database: PostgreSQL on Railway

Cache + Queues: Redis on Upstash (BullMQ)

File Storage: IPFS via Pinata

Blockchain: Stellar Testnet

- Horizon: <https://horizon-testnet.stellar.org>
- Soroban RPC: <https://soroban-testnet.stellar.org>
- Explorer: <https://stellar.expert/explorer/testnet>

Hosting

- Frontend: Vercel (free)
- Backend: Railway (free tier)
- Database: Railway PostgreSQL
- Redis: Upstash (free 10k commands/day)

Environment Variables

```
NODE_ENV=development
PORT=5000
FRONTEND_URL=http://localhost:5173
DATABASE_URL=postgresql://user:pass@host:5432/cryptwill
REDIS_URL=redis://default:password@host.upstash.io:6379
JWT_SECRET=minimum_32_character_secret_key_here
JWT_EXPIRES_IN=7d
REFRESH_TOKEN_SECRET=another_32_character_secret_here
REFRESH_TOKEN_EXPIRES_IN=30d
RESEND_API_KEY=re_xxxxxxxxxxxxxx
EMAIL_FROM=noreply@cryptwill.app
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your_auth_token
TWILIO_PHONE_NUMBER=+1234567890
STELLAR_NETWORK=testnet
STELLAR_HORIZON_URL=https://horizon-testnet.stellar.org
STELLAR_RPC_URL=https://soroban-testnet.stellar.org
POOL_WALLET_PUBLIC=G_your_pool_wallet_public_key
POOL_WALLET_SECRET=S_your_pool_wallet_secret_key
PINATA_API_KEY=your_pinata_api_key
PINATA_SECRET_KEY=your_pinata_secret_key
PINATA_JWT=your_pinata_jwt
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxxx
RAZORPAY_KEY_SECRET=your_razorpay_secret
RAZORPAY_WEBHOOK_SECRET=your_webhook_secret
```

Security Rules

- Passwords: bcrypt salt rounds 12
- JWT: 7 day expiry, refresh token 30 days
- File encryption: AES-256 client-side BEFORE upload — server never sees plaintext
- Key splitting: Shamir's — client-side ONLY — server never sees full private key
- Rate limiting: all auth endpoints (max 10 requests per 15 minutes)
- CORS: production domain only
- Input validation: express-validator on every endpoint
- Helmet: all security headers enabled
- OTP: 6 digits, expires 10 minutes, bcrypt hashed in DB

Coding Standards

- Async/await everywhere, no callbacks
- All errors: { success: false, error: "message" }
- All success: { success: true, data: {...} }
- Prisma for all DB operations — no raw SQL
- Zod schemas for all frontend form validation
- Never hardcode secrets — always process.env

SECTION 3 — APP FLOW (Every Screen and Route)

Route Map

PUBLIC ROUTES (no auth)

/	Landing page
/pricing	Pricing plans
/how-it-works	Product explainer
/faq	FAQ
/terms	Terms & Conditions
/privacy	Privacy Policy
/signup	Registration
/login	Login
/forgot-password	Password reset request
/reset-password/:token	Password reset form
/verify-email/:token	Email OTP verification
/checkin/instant/:token	One-click check-in from email (NO LOGIN NEEDED)
/beneficiary/invite/:token	Beneficiary accepts invitation
/guardian/invite/:token	Guardian accepts invitation

OWNER ROUTES (requires auth)

/dashboard	Main dashboard
/onboarding	First-time setup (5 steps)
/assets	Asset registry list
/assets/add	Add new asset
/assets/:id/edit	Edit asset
/will	Will builder
/will/deploy	Deploy smart contract
/will/preview	Full will preview
/beneficiaries	Beneficiary management
/beneficiaries/add	Add beneficiary
/guardians	Guardian management
/guardians/add	Add guardian
/checkin	Check-in page + history
/vault	Digital vault file manager
/vault/upload	Upload file to vault
/contract	Contract status + details
/notifications	Notification history
/settings/profile	Profile settings
/settings/security	Password + 2FA
/settings/kyc	KYC documents
/settings/notifications	Notification preferences

/settings/subscription	Plan + billing
BENEFICIARY PORTAL (separate auth)	
/beneficiary/login	Beneficiary login
/beneficiary/dashboard	Locked assets view
/beneficiary/vault	Vault files (locked)
/beneficiary/notifications	Notification history
GUARDIAN PORTAL (separate auth)	
/guardian/login	Guardian login
/guardian/dashboard	Responsibilities view
/guardian/vote/:contractId	Vote interface
/guardian/history	Vote history
ENTERPRISE PORTAL (separate auth)	
/enterprise/login	Enterprise login
/enterprise/dashboard	All clients
/enterprise/client/:id	Single client view
/enterprise/export	Compliance exports

Key Screen Specifications

LANDING PAGE (/)

Sections (top to bottom):

1. Navbar: Logo + How It Works + Pricing + Login + Get Started (CTA button)
2. Hero: Big headline "Your crypto shouldn't die with you." + subtext + two CTAs (Get Started Free / See How It Works) + subtle animated blockchain visual
3. Problem section: "\$140 Billion Lost" stat card + 3 reasons why existing solutions fail
4. How It Works: 4 steps with icons (Setup / Check-in / Guardian Confirms / Assets Transfer)
5. Features grid: 6 cards (Dead Man's Switch, Guardian Consensus, Digital Vault, Proof of Life, Shamir Security, Staged Release)
6. Pricing: 3 columns (Free / Pro / Enterprise)
7. FAQ: 6 accordion items
8. Footer: Logo + links + "Built on Stellar" badge

SIGNUP (/signup)

Fields: Full name, Email, Password, Confirm Password, Country, Phone, T&C checkbox, Age 18+ checkbox
Flow: Submit → OTP sent → /verify-email → on success → /onboarding

ONBOARDING (/onboarding) — 5 steps, progress bar

- Step 1: Personal details (DOB, country, phone)
- Step 2: KYC upload (select type + drag-drop file)
- Step 3: Bank declaration (bank name, account number masked, IFSC)
- Step 4: Choose plan (Free vs Pro cards with Razorpay on Pro)
- Step 5: Connect Freightier wallet + completion screen → /dashboard

MAIN DASHBOARD (/dashboard)

Layout: Fixed sidebar left + scrollable main content right

Sidebar items: Dashboard, My Assets, Will Builder, Beneficiaries, Guardians, Check-In, Digital Vault, Contract, Notifications, Settings

Top row cards:

- Assets Registered: [count]
- Beneficiaries: [count]
- Guardians Active: [X / Y]
- Contract: [Active / Not Deployed / Triggered]

Check-In Status card (prominent, center):

- Color-coded countdown: Green (> 10 days) Yellow (5 days) Red (2 days)
- Progress bar showing days elapsed
- "Check In Now" button (only when contract active)
- Last check-in: [date + Stellar Explorer link]

Proof of Life Chain: Last 6 tokens as timeline (date + tx hash clickable)

Recent Activity Feed: Last 10 events with timestamps

CHECK-IN PAGE (/checkin)

- Big countdown card with color status
- "Check In Now" button → Freighter opens → signs micro-transaction → Proof of Life minted → timer resets
- Success state: "You're checked in  Next due: [date]"
- History table: Date | Tx Hash | PoL Token | Method
- Reminder preferences: toggle SMS on/off, set reminder days

DIGITAL VAULT (/vault)

- Storage meter bar (used / total)
- File grid or list view
- Upload button → drag-drop zone appears
- Upload flow shows steps: Encrypting... Uploading to IPFS... Saving reference...
- Each file: name, size, lock icon (encrypted), assigned beneficiary, date
- Actions: view info, reassign, delete

GUARDIAN PORTAL (/guardian/dashboard)

Normal state: "You are a guardian for [Owner Name]. Status: Waiting." Vote required state:

- Red banner: "ACTION REQUIRED"
- "[Owner Name] has missed [X] check-ins. Last seen: [date]."
- Vote tally: "[X] Approve | [Y] Deny | [Z] Pending"
- Two large buttons: "CONFIRM — I believe they have passed" / "DENY — I believe they are alive"

- Optional note field

BENEFICIARY PORTAL (/beneficiary/dashboard)

Status banner: Waiting / Process Started / Transfer Complete Asset list: each with lock icon 

- Name | Type | Your share | Release date | Status: Locked Note: "These assets are secured and will transfer automatically." Vault files: file names listed with lock, "Key releases automatically"

SECTION 4 — NOTIFICATION SYSTEM (Complete Spec)

Stack

Events → BullMQ Queue (Redis/Upstash) → Worker → Resend (email) or Twilio (SMS) → Logged to DB

Why BullMQ Not Just node-cron

BullMQ persists jobs in Redis. If server restarts, no jobs are lost. For an inheritance system, losing a guardian notification because the server restarted is catastrophic. BullMQ gives retries, exponential backoff, and job persistence automatically.

Queue Names

```
EMAIL_QUEUE = 'email-notifications'
SMS_QUEUE = 'sms-notifications'
CHECKIN_MONITOR = 'checkin-monitor'
```

BullMQ Setup

```
import { Queue, Worker } from 'bullmq'
const connection = { url: process.env.REDIS_URL }

const emailQueue = new Queue('email-notifications', { connection })
const smsQueue = new Queue('sms-notifications', { connection })

// Add job with retry config
await emailQueue.add('job-name', { data }, {
  attempts: 3,
  backoff: { type: 'exponential', delay: 2000 },
  removeOnComplete: 100,
  removeOnFail: 200,
})

// Worker
const emailWorker = new Worker('email-notifications', async (job) => {
  await resend.emails.send({
    from: process.env.EMAIL_FROM,
```

```

    to: job.data.to,
    subject: job.data.subject,
    html: job.data.html,
  })
  // Log to DB: { userId, type, channel, status: 'SENT', sentAt: now }
}, { connection })

emailWorker.on('failed', (job, err) => {
  // Log to DB: { status: 'FAILED', failureReason: err.message }
})

```

One-Click Check-In Link (Technical Spec)

```

// Generate JWT for email check-in (no login needed)
const checkInToken = jwt.sign(
  { userId: user.id, purpose: 'checkin', contractId: contract.id },
  process.env.JWT_SECRET,
  { expiresIn: '5d' }
)
const checkInLink = `${process.env.FRONTEND_URL}/checkin/instant/${checkInToken}`

// Route – NO auth middleware
GET /api/checkin/instant/:token
→ Verify JWT
→ Check purpose === 'checkin'
→ Submit Stellar micro-transaction (pool wallet signs)
→ Mint Proof of Life token
→ Update contract: last_checkin_at, reset missed_checkin_count, update
next_checkin_due
→ Log check-in to DB
→ Queue confirmation email
→ Return success page (no redirect to login)

```

Daily Check-In Monitor Cron

```

// Runs every day at 9:00 AM
cron.schedule('0 9 * * *', async () => {
  const contracts = await prisma.contract.findMany({
    where: { status: 'ACTIVE' },
    include: { user: true }
  })

  for (const contract of contracts) {
    const daysSince = differenceInDays(new Date(), contract.lastCheckinAt)
    const daysUntilDeadline = contract.checkinIntervalDays - daysSince

    if (daysUntilDeadline === 5) {
      await emailQueue.add('reminder-early', { userId: contract.userId, contract
    })
  }
}

```

```

    }
    if (daysUntilDeadline === 2) {
      await emailQueue.add('reminder-urgent', { userId: contract.userId, contract
    })
      await smsQueue.add('reminder-urgent-sms', { phone: contract.user.phone,
contract })
    }
    if (daysUntilDeadline <= 0) {
      await handleMissedCheckin(contract)
    }
  }
})

async function handleMissedCheckin(contract) {
  await prisma.contract.update({
    where: { id: contract.id },
    data: { missedCheckinCount: { increment: 1 } }
  })

  // Alert owner
  await emailQueue.add('missed-checkin-owner', { contract })
  await smsQueue.add('missed-checkin-owner-sms', { contract })

  // After 3 misses: alert guardians
  if (contract.missedCheckinCount >= 2) { // 0-indexed, so 2 = 3rd miss
    await triggerGuardianAlert(contract)
  }
}

```

Complete Notification Events List

AUTH

1. Email OTP on signup → Email → immediate
2. Welcome email after OTP verified → Email → immediate
3. Password reset OTP → Email → immediate

CHECK-IN (Most Critical)

4. Reminder Early (Day 25 of 30) → Email only Subject: "🕒 Your CryptWill check-in is due in 5 days" Body: "Hey [Name], your check-in is due on [date]. Click to check in now — takes 5 seconds." CTA: One-click check-in link (JWT signed)
5. Reminder Urgent (Day 28) → Email + SMS Email subject: "🔥 2 days left — check in now" SMS: "CryptWill: Check in now or guardians notified in 2 days. [link]"
6. Reminder Final (Day 29, 12 hrs before) → Email + SMS Email subject: "🚫 Final notice — check in before midnight" SMS: "CryptWill URGENT: Check in TODAY or inheritance starts tomorrow. [link]"
7. Check-in Confirmed → Email only Subject: "✅ Check-in confirmed" Body: "Recorded at [time]. PoL token: [Explorer link]. Next due: [date]."

8. Check-in Missed — Owner Alert → Email + SMS Subject: " ⚠️ You missed your check-in" Body: "Guardians will be notified. Click here to check in NOW and cancel." CTA: Emergency check-in link (still valid for 3 days after deadline)

GUARDIAN

9. Guardian Invitation → Email only Subject: "[Owner Name] has named you as a Guardian on CryptWill" Body: Explains responsibility, link to accept
10. Guardian Accepted (to owner) → Email only Subject: "Guardian [Name] has accepted their role"
11. Guardian Alert — Action Required (CRITICAL) → Email + SMS simultaneously Subject: " ⚠️ Guardian Action Required — [Owner Name] has been unreachable" Body: How many check-ins missed, vote portal link, 14-day window SMS: "CryptWill: [Owner Name] missed check-ins. Guardian vote needed. [link]"
12. Vote Update (to all guardians) → Email Subject: "Guardian vote update — [X of Y] confirmed" Body: Current tally, how many more needed
13. Quorum Reached → Email + SMS (all guardians + beneficiaries) Subject: "Guardian consensus reached — inheritance begins"
14. Guardian Denied → Email (all guardians) Subject: "A Guardian has denied — 48hr hold" Body: Guardian name, their note, what happens next

BENEFICIARY

15. Beneficiary Invitation → Email only Subject: "[Owner Name] has added you as a beneficiary on CryptWill" Body: What they'll receive, link to create account + preview
16. Inheritance Started → Email + SMS Subject: "Important: [Owner Name]'s digital assets" Body: Assets they'll receive, first transfer date, no action needed
17. Asset Received → Email + SMS Subject: " ✅ Assets received from [Owner Name]'s estate" Body: What transferred, Stellar Explorer tx link, vault access if applicable

SYSTEM

18. Contract Deployed → Email Subject: " ✅ Your CryptWill contract is live" Body: Contract address, Explorer link, first check-in date
19. Subscription Confirmed → Email Subject: " ✅ You're on CryptWill [Pro/Enterprise]" Body: Features unlocked, next billing date
20. Subscription Expiring → Email (7 days before) Subject: "Your CryptWill Pro renews in 7 days"

SECTION 5 — BACKEND SCHEMA (Prisma)

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum KYCStatus { NOT_SUBMITTED SUBMITTED VERIFIED REJECTED }
enum Plan { FREE PRO ENTERPRISE }
enum ContractStatus { NOT_DEPLOYED ACTIVE TRIGGERED EXECUTING EXECUTED CANCELLED }
enum AssetType { STELLAR_WALLET CUSTOM_TOKEN NFT DOCUMENT }
enum GuardianStatus { INVITED ACTIVE DECLINED REMOVED }
enum BeneficiaryStatus { INVITED ACTIVE DECLINED }
enum VoteType { APPROVE DENY }
enum NotificationType {
  EMAIL_OTP WELCOME PASSWORD_RESET
  CHECKIN_REMINDER_EARLY CHECKIN_REMINDER_URGENT CHECKIN_REMINDER_FINAL
  CHECKIN_CONFIRMED CHECKIN_MISSED
  GUARDIAN_INVITE GUARDIAN_ACTIVATED GUARDIAN_ALERT
  GUARDIAN_VOTE_UPDATE GUARDIAN_QUORUM_REACHED GUARDIAN_DENIED
  BENEFICIARY_INVITE BENEFICIARY_INHERITANCE_STARTED BENEFICIARY_ASSET_RECEIVED
  CONTRACT_DEPLOYED SUBSCRIPTION_CONFIRMED SUBSCRIPTION_EXPIRING
}
enum NotificationChannel { EMAIL SMS }
enum NotificationStatus { QUEUED SENT FAILED }
enum SubscriptionStatus { ACTIVE CANCELLED EXPIRED PAST_DUE }

model User {
  id          String    @id @default(uuid())
  email       String    @unique
  passwordHash String
  fullName    String
  phone       String?
  dateOfBirth DateTime?
  country     String?
  walletAddress String?
  kycStatus   KYCStatus @default(NOT_SUBMITTED)
  kycDocumentUrl String?
  kycDocumentType String?
  bankDeclaration String?
  plan        Plan      @default(FREE)
  isVerified  Boolean    @default(false)
  isOnboarded Boolean    @default(false)
  termsAcceptedAt DateTime?
  termsVersion String?
  otpCode     String?
  otpExpiresAt DateTime?
  refreshToken String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

```

```

    assets          Asset[]
    beneficiaries   Beneficiary[]
    guardians       Guardian[]
    contract        Contract?
    vaultFiles      VaultFile[]
    notifications   Notification[]
    subscription    Subscription?
    checkIns        CheckIn[]
    termsAcceptances TermsAcceptance[]
  }

  model Asset {
    id              String      @id @default(uuid())
    userId          String
    user            User        @relation(fields: [userId], references: [id],
onDelete: Cascade)
    assetName       String
    assetType       AssetType
    walletAddress   String?
    tokenCode       String?
    estimatedValueUsd Decimal?
    ipfsHash        String?
    encryptionKeyRef String?
    releaseDay      Int          @default(0)
    specialInstructions String?
    isActive        Boolean      @default(true)
    createdAt       DateTime      @default(now())
    updatedAt       DateTime      @updatedAt

    assignments     AssetBeneficiaryAssignment[]
    vaultFiles      VaultFile[]
  }

  model Beneficiary {
    id              String      @id @default(uuid())
    userId          String
    user            User        @relation(fields: [userId], references:
[id], onDelete: Cascade)
    fullName        String
    email           String
    phone           String?
    walletAddress   String?
    verificationToken String?
    status          BeneficiaryStatus @default(INVITED)
    passwordHash    String?
    createdAt       DateTime      @default(now())
    updatedAt       DateTime      @updatedAt

    assignments     AssetBeneficiaryAssignment[]
    vaultFiles      VaultFile[]
  }

  model AssetBeneficiaryAssignment {

```



```

    id                String          @id @default(uuid())
    assetId           String
    asset             Asset           @relation(fields: [assetId], references: [id],
onDelete: Cascade)
    beneficiaryId     String
    beneficiary       Beneficiary @relation(fields: [beneficiaryId], references: [id],
onDelete: Cascade)
    percentage        Int             @default(100)
    notes             String?
    createdAt         DateTime        @default(now())

    @@unique([assetId, beneficiaryId])
}

model Guardian {
    id                String          @id @default(uuid())
    userId           String
    user             User            @relation(fields: [userId], references: [id],
onDelete: Cascade)
    fullName         String
    email            String
    phone            String?
    inviteToken      String?
    status           GuardianStatus @default(INVITED)
    passwordHash     String?
    acceptedAt       DateTime?
    createdAt        DateTime        @default(now())
    updatedAt        DateTime        @updatedAt

    votes            GuardianVote[]
}

model GuardianVote {
    id                String          @id @default(uuid())
    contractId       String
    contract         Contract @relation(fields: [contractId], references: [id])
    guardianId       String
    guardian         Guardian @relation(fields: [guardianId], references: [id])
    vote             VoteType
    notes            String?
    votedAt          DateTime        @default(now())

    @@unique([contractId, guardianId])
}

model Contract {
    id                String          @id @default(uuid())
    userId           String          @unique
    user             User            @relation(fields: [userId], references: [id],
onDelete: Cascade)
    contractAddress   String?
    network           String          @default("testnet")
    status            ContractStatus @default(NOT_DEPLOYED)
    checkinIntervalDays Int          @default(30)

```

```

guardianQuorum      Int           @default(3)
missedCheckinCount  Int           @default(0)
deployedAt          DateTime?
lastCheckinAt       DateTime?
nextCheckinDue      DateTime?
triggerStartedAt    DateTime?
executedAt          DateTime?
stellarTxHash       String?
createdAt           DateTime      @default(now())
updatedAt           DateTime      @updatedAt

checkIns            CheckIn[]
votes               GuardianVote[]
}

model CheckIn {
  id                String        @id @default(uuid())
  contractId        String
  contract          Contract      @relation(fields: [contractId], references: [id])
  userId            String
  user              User          @relation(fields: [userId], references: [id])
  transactionHash   String
  proofOfLifeTokenId String?
  stellarExplorerUrl String?
  method            String        @default("manual")
  checkedInAt       DateTime      @default(now())
}

model VaultFile {
  id                String        @id @default(uuid())
  userId            String
  user              User          @relation(fields: [userId], references: [id],
onDelete: Cascade)
  assetId           String?
  asset             Asset?        @relation(fields: [assetId], references: [id])
  assignedBeneficiaryId String?
  beneficiary        Beneficiary? @relation(fields: [assignedBeneficiaryId],
references: [id])
  fileName          String
  fileSizeBytes     Int
  fileType          String
  ipfsHash          String
  isEncrypted        Boolean      @default(true)
  encryptionKeyRef   String?
  uploadedAt        DateTime      @default(now())
}

model Subscription {
  id                String        @id @default(uuid())
  userId            String        @unique
  user              User          @relation(fields: [userId],
references: [id], onDelete: Cascade)
  plan              Plan          @default(FREE)
  razorpaySubscriptionId String?

```

```

    razorpayCustomerId      String?
    status                  SubscriptionStatus @default(ACTIVE)
    startedAt               DateTime          @default(now())
    expiresAt               DateTime?
    cancelledAt             DateTime?
    createdAt               DateTime          @default(now())
    updatedAt               DateTime          @updatedAt
  }

  model Notification {
    id                      String            @id @default(uuid())
    userId                  String
    user                    User              @relation(fields: [userId], references:
[id], onDelete: Cascade)
    type                    NotificationType
    channel                  NotificationChannel
    status                  NotificationStatus @default(QUEUED)
    recipient                String
    subject                  String?
    body                     String
    metadata                 Json?
    sentAt                  DateTime?
    failureReason            String?
    createdAt               DateTime          @default(now())
  }

  model TermsAcceptance {
    id                      String            @id @default(uuid())
    userId                  String
    user                    User              @relation(fields: [userId], references: [id], onDelete:
Cascade)
    version                 String
    acceptedAt              DateTime          @default(now())
    ipAddress                String?
    userAgent                String?
  }

```

SECTION 6 — UI/UX BRIEF

Design Identity

CryptWill feels like a premium fintech product that takes digital security seriously. Dark theme, clean lines, no clutter. If Linear and Coinbase had a baby that also cared about death — that's CryptWill.

Colors

```

Background primary:    #0A0A0F (near black)
Background secondary:  #111118 (cards, sidebar)
Background elevated:    #1A1A24 (modals, dropdowns)

```

```
Border:                #2A2A3A (subtle card borders)
Brand blue:            #4F6EF7 (primary CTA, links, active states)
Brand blue hover:      #3D5CE5
Brand glow:            rgba(79, 110, 247, 0.15) (for card glow effects)
Success green:         #22C55E
Warning yellow:        #F59E0B
Danger red:            #EF4444
Text primary:          #F0F0F5 (headings)
Text secondary:        #9090A0 (subtitles, labels)
Text muted:            #606070 (placeholders, helpers)
```

Typography

```
Font family:  'Inter', sans-serif (import from Google Fonts)
Heading 1:    48px / 700 weight / -0.02em tracking
Heading 2:    32px / 600 weight / -0.01em tracking
Heading 3:    24px / 600 weight
Body large:   18px / 400 weight / 1.6 line height
Body:         16px / 400 weight / 1.6 line height
Small:        14px / 400 weight
Label:        12px / 500 weight / 0.05em tracking / UPPERCASE
```

Spacing System (use these values only)

4px, 8px, 12px, 16px, 24px, 32px, 48px, 64px, 96px

Border Radius

```
Small (buttons, inputs, badges): 8px
Medium (cards): 12px
Large (modals, panels): 16px
Full (pills, avatars): 9999px
```

Component Styles

Cards

```
background: #111118
border: 1px solid #2A2A3A
border-radius: 12px
padding: 24px
/* Hover state - subtle glow */
box-shadow: 0 0 0 1px rgba(79, 110, 247, 0.3)
transition: box-shadow 0.2s ease
```

Primary Button

```
background: #4F6EF7
color: white
border-radius: 8px
padding: 12px 24px
font-weight: 600
font-size: 16px
/* Hover */
background: #3D5CE5
transform: translateY(-1px)
box-shadow: 0 4px 20px rgba(79, 110, 247, 0.4)
```

Input Fields

```
background: #0A0A0F
border: 1px solid #2A2A3A
border-radius: 8px
padding: 12px 16px
color: #F0F0F5
font-size: 16px
/* Focus */
border-color: #4F6EF7
outline: none
box-shadow: 0 0 0 3px rgba(79, 110, 247, 0.15)
```

Sidebar

```
background: #111118
border-right: 1px solid #2A2A3A
width: 240px
height: 100vh
position: fixed
/* Active nav item */
background: rgba(79, 110, 247, 0.1)
border-left: 2px solid #4F6EF7
color: #4F6EF7
```

Status Badges

```
/* Active / Success */
background: rgba(34, 197, 94, 0.1)
color: #22C55E
border: 1px solid rgba(34, 197, 94, 0.2)
```

```
/* Warning */  
background: rgba(245, 158, 11, 0.1)  
color: #F59E0B  
border: 1px solid rgba(245, 158, 11, 0.2)  
  
/* Danger */  
background: rgba(239, 68, 68, 0.1)  
color: #EF4444  
border: 1px solid rgba(239, 68, 68, 0.2)  
  
/* Locked / Neutral */  
background: rgba(96, 96, 112, 0.1)  
color: #9090A0  
border: 1px solid rgba(96, 96, 112, 0.2)
```

Page Layout

Dashboard Layout

```
[Sidebar 240px fixed] [Main content – remaining width]  
                        [Top bar – user avatar + notifications bell]  
                        [Page content – max-width 1200px, centered]
```

Auth Pages (Login/Signup)

```
[Left half – full height brand panel with CryptWill logo + tagline + visual]  
[Right half – form centered vertically]
```

Mobile (below 768px)

- Sidebar collapses to bottom navigation bar
- Cards stack full width
- Sidebar becomes a drawer

Specific UI Components

Check-In Status Card

- Circular countdown ring (SVG) showing days remaining
- Color transitions: green → yellow → red as deadline approaches
- Pulsing red animation when overdue
- "Check In Now" button appears prominently

Proof of Life Timeline

- Horizontal scrollable row of check-in tokens

- Each token: small card with date, check number,  icon, clickable → Stellar Explorer
- Last item glows blue (most recent)

Staged Release Timeline (Will Builder)

- Vertical timeline with dots and connecting line
- Each node: day number, icon, asset list at that release point
- Nodes turn green as they complete post-death

Guardian Vote Card (Guardian Portal)

- Split layout: left = owner info + missed check-in count, right = vote buttons
- Vote tally bar shows live approve vs deny
- Confirm button: red with warning icon
- Deny button: outlined white

Vault File Grid

- File card: icon by type (PDF/image/doc), name, size, lock icon
- Lock icon in top right (gold when locked, open when accessible)
- Upload card always first in grid (dashed border, + icon)

Animations

- Page transitions: fade in 200ms
- Card hover: 150ms ease
- Button hover: 150ms ease + slight lift
- Check-in success: brief confetti burst
- Status changes: smooth color transition 300ms
- Loading states: skeleton shimmer (gray pulse)

Icons

Use lucide-react exclusively. Key icons:

- Shield (security/contract)
- Heart (check-in / alive)
- Users (guardians)
- User (beneficiary)
- Lock / Unlock (vault)
- Clock (deadline)
- CheckCircle (confirmed)
- AlertTriangle (warning)
- XCircle (danger)
- TrendingUp (assets)
- Link (Stellar Explorer links)
- Download (shard download)

Inspiration References

- Linear.app (dark theme, sidebar, clean cards)
 - Coinbase.com (trust, security-forward design)
 - Vercel dashboard (minimalist, functional)
 - Stripe dashboard (data clarity)
-